

AUTONOMOUS AI SYSTEM

Continuous Executive Coding Agent

Jack Jin



v2.1.0

Latest Release

April 2024

Project Timeline

AI That Works Like a Persistent Entity

Reactive Model

Traditional AI agents wait for prompts. You type, they respond. Every interaction requires human initiation.

- ✗ Requires constant human attention
- ✗ Sequential task processing only
- ✗ No autonomous prioritization

Autonomous Model

You drop goals into a queue, and it works through them autonomously — selecting highest priority, breaking complex goals into steps, retrying with different strategies.

- ✓ Runs 24/7 via PM2
- ✓ Priority-based work selection
- ✓ Async communication via markdown

The Result

An AI that operates more like a **junior developer with a task board** than a chatbot waiting for instructions.

Core Philosophy:

"Flip the model: from reactive to proactive.
From prompt-response to goal-achievement."

CORE MECHANISM

The Executive Loop

8-Phase Continuous Operation



Work Hierarchy

Goal → Steps → Contracts



Goal Lifecycle

drafts → ondeck → in-progress → completed



Sleep Behavior

Only when queue is empty

Three-Layer Architecture



Agentic Layer

`src/agentic/`

AI decisions — work selection, strategy, diagnosis, prompt building

Uses LLM

Intelligent

Decision-making



Deterministic Layer

`src/deterministic/`

Mechanical operations — file I/O, health checks, state updates, Notion reporting

No LLM

Reliable

Predictable



Core Layer

`src/core/`

Orchestration — loop coordination, types, logging

Foundation

Types

Logging



Two-Repository Design

`continuous-agent/`

The brain — agent infrastructure, executive loop, worker spawner



`ai-sandbox/`

The output — all worker outputs, isolated project directories

Key Rule: Workers NEVER write to agent codebase. Enforced by Constitution.



Identity Layer (v2.0)



Gmail

Inbox checking for goals



Discord

Completion/blocked notifications

All opt-in, disabled by default

v1.0 Foundation

The Core Infrastructure Launch



Executive Loop

8-phase continuous loop: health check, input processing, work selection, contract creation, execution, validation, state updates, sleep/continue.

✓ Runs 24/7 via PM2



Contract System

Scoped work agreements with prompt, allowed tools, Definition of Done, and turn budgets. Each contract spawns an isolated Claude worker session.

✓ Isolated execution per task



Verifier System

Deterministic post-execution validation: git-clean, node-build, docs-checklist, reference-integrity. Ensures output quality before marking complete.

✓ Automated quality gates



Append-Only Ledgers

JSONL audit trail: work-ledger, capability-ledger, daily executive logs, per-contract worker logs. Immutable history for debugging and analysis.

✓ Full traceability



Capability Tracking

YAML registries (technical, delivery, functional) with confidence scoring. +10 on PASS, -15 on FAIL. Self-improving capability awareness.

✓ Learns from execution



The Constitution

8 immutable hard limits: spending caps, no deletions, no credential exposure, mandatory logging, 10-retry minimum. Safety guardrails.

✓ Agent cannot override

v1.1-1.3 Enhancements

Incremental Improvements & Infrastructure Hardening

v1.1

Jan 25, 2024

Intelligence Layer

- ▶ **Incremental execution** — Goals >100 turns auto-break into 2-5 steps
- ▶ **Self-enhancement workflow** — [SELF-ENHANCE] goals route to agent codebase
- ▶ **Strategy rotation** — Retries cycle through different approaches
- ▶ **Agentic diagnosis** — LLM analyzes root cause after 3+ failures
- ▶ **needs-you.md** — Async communication with response tags

v1.2

Jan 28, 2024

Goal Management

- ▶ **Goal bundles** — Directories with PROMPT.md (YAML frontmatter)
- ▶ **Workspace lifecycle** — drafts/ → ondeck/ → in-progress/ → completed/
- ▶ **STEPS.json** — Machine-readable step tracking (source of truth)
- ▶ **Notion reporting** — Milestone events, daily/weekly summaries
- ▶ **Project memory** — capabilities/project-memory.yml records lessons

v1.3

Feb 4, 2024

Security & Auth

- ▶ **Three-tier credential system** — Physical separation of executive/worker/app tiers
- ▶ **Leak detection** — Validates no cross-tier contamination
- ▶ **OAuth-first authentication** — CLAUDE_CODE_OAUTH_TOKEN via Pro/Max
- ▶ **Tier 3 format helpers** — Converts to dotenv, JSON, shell, docker-compose

v2.0 Identity & Intelligence

Major Platform Upgrade

Agent Identity System

The agent now has its own communication presence — it can receive goals via email and push notifications to Discord.

 **Gmail**

Checks inbox for new goals, approvals, priority changes

 **Discord**

Sends completion/blocked notifications via webhooks

All opt-in, disabled by default via feature flags

Execution Patterns

Four worker execution modes, specified per-goal in PROMPT.md frontmatter:

plan-then-execute

Default. Research then build.

loop-until-progress

Keep iterating while making gains.

plan-mode

Read-only analysis, no code changes.

deterministic-pipeline

Fixed multi-step sequence.

Skills & Playbooks

Separated reusable knowledge into two libraries:

Skills

Atomic tool/API how-tos (e.g., "How to use Supabase Auth")

Playbooks

Goal-oriented workflows (e.g., "Build a React dashboard")

Track Record System: Skills carry execution history (total executions, success rate, confidence, maturity)

Dashboard & Observability

- ✓ **Dashboard Writer** — Generates workspace/dashboard-data.json
- ✓ **Dashboard UI** — Next.js/React read-only dashboard
- ✓ **Pipeline Executor** — Deterministic multi-step execution

v2.1 Multi-Vendor Support

Worker Tier Abstraction — Claude, Codex, Kimi

Vendor Abstraction Layer

AgentWorkerProvider

Full agentic execution — tools, file editing, code execution

ChatCompletionProvider

Simple text-in/text-out LLM calls — breakdown, diagnosis

Three Worker Backends



Claude (Default)

Claude Agent SDK query()

OAuth



OpenAI Codex

Codex SDK threads

CLI Login



Kimi K2.5

Wire SDK or CLI stream-json

Dual Mode

Multi-Vendor Comparison Results

Same prompt executed across all vendors: "Build a personal finance dashboard in React"

1

Codex

Best architecture, working dark mode

115

/130

2

Kimi CLI

Richest interactivity

92

/130

3

Claude

Most concise, best LOC/feature ratio

88

/130

4

Kimi Wire

Largest bundle, some UI bugs

87

/130

 Live at: jackzhaojin.github.io/ai-sandbox

v2.2 Cloud Database Migration

Moving Mutable State from Local Files to Cloud

The Vision

Move all **mutable agent operational data** out of local flat files and into a cloud database. Local files become optional cache/fallback — the cloud DB becomes the source of truth.

Queryability

Aggregate, filter, analyze historical data

Durability

No data loss on machine wipe

Concurrent Access

Multiple processes, no corruption

Observability

Real-time dashboards & queries

StorageProvider Abstraction

LocalFileProvider

Wraps current fs-based behavior (fallback)

CloudDbProvider

Supabase or Cosmos DB implementation

Feature Flag: `V2_CLOUD_DB=true` — revert to local with zero breakage

What Moves to Cloud



work_ledger

ledgers/work-ledger.jsonl



executive_logs

ledgers/executive-*.log



contract_events

workspace/**/CONTRACTS.jsonl



goal_steps

workspace/**/STEPS.json



evolution_log

learning/evolution-log.jsonl



project_registry

workspace/project-registry.yml

Stays Local



constitution.md (immutable)



PROMPT.md files (human-authored)



.env.* files (credentials)



capabilities/* .yml (skills)

What the Agent Can Do Today



24/7 Operation

- ✓ Runs continuously via PM2
- ✓ Hot reload without restart
- ✓ Sleeps only when queue empty
- ✓ Auto-recovery on failures



Goal Management

- ✓ Priority-based queue (P0-P4)
- ✓ Auto-break complex goals into steps
- ✓ Lifecycle: drafts → ondeck → in-progress → completed
- ✓ YAML frontmatter for metadata



Multi-Vendor Workers

- ✓ Claude Agent SDK (default)
- ✓ OpenAI Codex SDK
- ✓ Kimi Wire + CLI modes
- ✓ Per-goal vendor override



Validation & Quality

- ✓ Deterministic verifiers
- ✓ Strategy rotation on retry
- ✓ Agentic diagnosis after 3+ failures
- ✓ 10-retry minimum before blocking



Communication

- ✓ Async via needs-you.md
- ✓ Response tags: [APPROVED], [DECISION], [INFO], [SKIP]
- ✓ Gmail inbox checking (opt-in)
- ✓ Discord notifications (opt-in)



Deployment

- ✓ GitHub Pages auto-deployment
- ✓ Recursive project discovery
- ✓ Per-project --base path injection
- ✓ Landing page grouped by date

The Constitution

8 Immutable Hard Limits

1

Spending Caps

No spending beyond **\$20/month per service**. Prevents runaway costs.

2

No Deletions

No permanent deletions — **archive/soft-delete only**. Everything recoverable.

3

No Publishing

No external publishing **without approval**. Human gate on public releases.

4

No Credentials

No credential exposure. **Three-tier system** with leak detection.

5

No Access Expansion

No access control expansion. **Cannot grant itself more permissions**.

6

No Agent Output

No output in agent codebase. **All outputs to ai-sandbox/**.

7

Mandatory Logging

All activity must be logged. **Append-only ledgers** for audit trail.

8

10 Retries Minimum

10 retries minimum before blocking. **Never gives up easily**.



Agent cannot override these limits. They are hard-coded safety boundaries.

THE FUTURE OF AUTONOMOUS AI

Building the Future

From reactive chatbots to proactive agents that work like team members

The Continuous Executive Agent represents a **fundamental shift** in how we interact with AI — not waiting for prompts, but collaborating through goals.



Autonomous



24/7 Operation



Self-Improving

- Continuous Executive Agent v2.1.0